



Introducing Automatically Derived Subproblem Relaxations in a Logic-Based Benders Decomposition Solver

Sachin Rajendran¹✉, Stephen Maher², and Elina Rönnberg¹

¹ Department of Mathematics, Linköping University, Linköping, Sweden
sachin.rajendran@liu.se

² GAMS Software GmbH, Frechen, Germany

Abstract. Logic-based Benders decomposition (LBBD) effectively combines mixed integer programming and constraint programming to solve difficult optimisation problems. While the decomposition exploits different solution and modelling paradigms, the separation destroys problem information connecting master and subproblem decisions. Alongside cut generation, a standard way of re-introducing this connection is through the addition of valid inequalities in the master problem. These inequalities typically constitute subproblem relaxations and they are usually problem specific and derived by hand. This paper proposes a general approach for deriving valid inequalities and demonstrates how they can be applied to cumulative and disjunctive constraints. In an effort to develop a general purpose LBBD solver, the general derivation of valid inequalities have been implemented as part of a new cumulative constraint handler for SCIP and the effectiveness is evaluated on two variants of scheduling problems. The computational experiments show the potential of applying these general valid inequalities to the master problem.

Keywords: logic-based Benders decomposition · generic valid inequalities · implication structure · SCIP

1 Introduction

Logic-Based Benders Decomposition (LBBD) has been successfully applied to a variety of applications in transportation, production management, supply chain, computing, and telecommunications [5]. Computationally effective implementations of LBBD commonly rely on problem-specific acceleration strategies such as cut-strengthening techniques, customised cuts, and adding subproblem-based valid inequalities to the master problem. Typically, such valid inequalities constitute a relaxation of a set of subproblem constraints and their variables, or expressed in terms of master problem variables but hand-crafted for a specific subproblem structure [5]. In this paper we introduce a generalised form of subproblem relaxations, expressed in terms of master problem variables. Preliminary computational experiments are performed to assess the potential of including such a subproblem relaxation. Further, we exploit the property of Linear

Programming (LP) relaxations in Constraint Integer Programming (CIP) and investigate the effect of the same relaxation in the subproblem for cut generation.

1.1 Preliminaries

Our work addresses a relatively general problem structure with implication constraints. In the following, the notion of an item should be interpreted in its most wide sense, e.g. a job, a vehicle, a product, or a patient. Let the set J index items and let the set S index subproblems. A subproblem $s \in S$ corresponds to an independent facility, machine, segment of a timeline or anything that an item can be assigned to. As decision variables, let x_{js} be a binary variable indicating if item j is assigned to subproblem s or not, and let y_{js} be defined such that it can belong to any type of domain $\mathcal{D}_{js}^s$ if the corresponding $x_{js} = 1$, and that it has to be zero otherwise, $j \in J$, $s \in S$. To model the implication structure, we use the notation $(y_{js})_{j \in J | x_{js}=1}$ to include the variables $y_{js} \in \mathcal{D}_{js}^s$ for each subproblem $s \in S$ where $x_{js} = 1$, and exclude them otherwise. Also let $x = (x_{js})_{j \in J, s \in S}$ and $y = (y_{js})_{j \in J, s \in S}$. Before decomposition, the problem is of the form

$$[\text{P}] \quad \min \quad h(x) + g(y) \quad (1)$$

$$\text{s.t.} \quad C^M(x), \quad (2)$$

$$C_s^s((y_{js})_{j \in J | x_{js}=1}), \quad s \in S, \quad (3)$$

$$y_{js} \in \mathcal{D}_{js}^s, \quad j \in J, s \in S \mid x_{js} = 1, \quad (4)$$

$$y_{js} = 0, \quad j \in J, s \in S \mid x_{js} = 0, \quad (5)$$

$$x_{js} \in \{0, 1\}, \quad j \in J, s \in S. \quad (6)$$

The reason for addressing this particular problem structure is that it captures many of the problems to which LBBDD has been successfully applied, see e.g. [2, 3, 6, 7, 9]. Thus, effectively handling such problems in a generic LBBDD solver, accelerated by general purpose techniques, has wide-reaching benefits. This paper is part of a research project towards implementing such a solver in SCIP [4] and its purpose is to complement the existing initiative with the NUTMEG solver [10] by addressing this more specific structure. Both NUTMEG and our ongoing work are based on the branch-and-check form of the LBBDD algorithm and use a Mixed-Integer Programming (MIP) formulation in the master problem. In the subproblem, NUTMEG elegantly leverages the conflict analysis capabilities of lazy clause generation Constraint Programming (CP) solvers. This means it is designed to handle general problem structures where the objective depends only on master problem variables. One property of the NUTMEG implementation is that the complete model is handled in the subproblem. This makes it less suitable for problems with the type of implication structure introduced above, where there are known benefits from handling them explicitly, see e.g. [9].

1.2 Contributions and Outline

The current section is concluded with a brief review of the use of subproblem relaxations. Section 2 provides the decomposition of [P] and a proposition stating the general valid inequalities that can be used as subproblem relaxation constraints in the master problem. Section 3 presents a general type of scheduling problem structure where the subproblem either contains cumulative or disjunctive constraints. For this structure, we show how to use common relaxations of the subproblem constraints as an input to our general subproblem relaxation in the master problem. We also show how these generically-derived valid inequalities generalises commonly used hand-crafted ones, confirming the potential of our inequalities. Section 4 details how the relaxation of the subproblem constraints are implemented in a new constraint handler that is used with the Benders decomposition framework in SCIP [11]. Further, this constraint handler generates valid inequalities for either the master problem or subproblem. The code is publicly available in https://gitlab.liu.se/eliro15/SCIP_LBBD_cumulative. Section 5 shows and explains the computational results for some specific models and publicly available instances from the literature: cumulative facility scheduling [6] and single-facility scheduling with a segmented timeline [12], both with makespan and fixed costs objectives.

1.3 Literature Review

Without aiming for completeness, this review provides some examples of subproblem relaxations derived from the subproblem constraints. For cumulative facility scheduling with fixed costs, the so-called energy relaxation can be used [6]. In this problem, tasks are assigned to facilities and then scheduled on the respective facilities, complying with task-specific release times and deadlines, and capacity restrictions stated by a cumulative constraint. The energy relaxation is hand-crafted from the cumulative constraint. For each possible interval between a release time-deadline pair, it states that the claimed capacity of tasks in this interval cannot exceed the total available amount of capacity in the interval.

Another important problem structure is single machine scheduling with sequence-dependent setup times and multiple time windows [8], which is a generalisation of single-facility scheduling with segmented time line [1]. In this problem, jobs are assigned to segments of a time line and then scheduled within the segment by a disjunctive constraint, adhering to job-and-segment-dependent release times and deadlines. For this problem, a segment relaxation is hand-crafted from the disjunctive constraint and for each segment, the total processing time of the assigned jobs are restricted to at most the length of the segment. This type of relaxation is also used for similar scheduling problems such as in [7].

Examples of vehicle routing applications include [13], where a condition on the total travel time in each subproblem is used. Alternatively, [2] uses clique inequalities representing when a vehicle cannot service a subset of customers.

2 Decomposition and Subproblem Relaxation

The problem is decomposed such that the master problem and subproblems contain the variables x_{js} and y_{js} , respectively, $j \in J$, $s \in S$. Assuming $g(y)$ is bounded from below by $\hat{g}$, the master problem in a given iteration is

$$\begin{aligned}
 \text{[MP]} \quad & \min \quad h(x) + \varphi \\
 & \text{s.t.} \quad \mathcal{C}^M(x), \\
 & \quad \mathcal{B}_\omega(x, \varphi), \quad \omega \in \Omega, \\
 & \quad \text{[Subproblem relaxation]} \\
 & \quad x_{js} \in \{0, 1\}, \quad j \in J, \quad s \in S, \\
 & \quad \varphi \geq \hat{g},
 \end{aligned}$$

where Ω is a set of all generated Benders cuts and $\mathcal{B}_\omega(x, \varphi)$, $\omega \in \Omega$, is a bounding function. Given a solution, $\hat{x} = (\hat{x}_{js})_{j \in J, s \in S}$, the subproblem is

$$\begin{aligned}
 \text{[SP}(\hat{x})] \quad & \min \quad g(y) \\
 & \text{s.t.} \quad \mathcal{C}_s^S((y_{js})_{j \in J | \hat{x}_{js}=1}), \quad s \in S, \\
 & \quad y_{js} \in \mathcal{D}_{js}^S, \quad j \in J, \quad s \in S \mid \hat{x}_{js} = 1.
 \end{aligned}$$

Note that when $\hat{x}_{js} = 0$ holds for a master problem variable, the corresponding variable $\hat{y}_{js} = 0$ is not present in the subproblem constraints, $j \in J$, $s \in S$.

We introduce subproblem relaxations on a specific form, given by the following proposition. For the proposition to be applicable, a required input is a model for the subproblem, or a relaxation of the subproblem, where at least a part of the formulation consists of linear inequalities. Such a model can be used both in the proposition and to formulate a relaxed subproblem for generating cuts.

Proposition 1. *For each $s \in S$, let the constraints*

$$\sum_{j \in J} a_{jsu}^{\text{LEQ}} w_{js} \leq b_{su}^{\text{LEQ}}, \quad u \in U, \quad (7) \quad \mathcal{C}_s^L((w_{js})_{j \in J | x_{js}=1}), \quad (9)$$

$$w_{js} \in \mathcal{D}_{js}^L, \quad j \in J \mid x_{js} = 1, \quad (10)$$

$$\sum_{j \in J} a_{jsv}^{\text{GEQ}} w_{js} \geq b_{sv}^{\text{GEQ}}, \quad v \in V, \quad (8) \quad w_{js} = 0, \quad j \in J \mid x_{js} = 0, \quad (11)$$

$$x_{js} \in \{0, 1\}, \quad j \in J, \quad s \in S, \quad (12)$$

provide a relaxation of the feasible set described by constraints (3)–(6) in [P]. Then,

$$\sum_{j \in J} a_{jsu}^{\text{MIN}} x_{js} \leq b_{su}^{\text{LEQ}}, \quad \text{with } a_{jsu}^{\text{MIN}} = \min_{s.t.} (9) - (11) a_{jsu}^{\text{LEQ}} w_{js}, \quad u \in U, \quad (13)$$

$$\sum_{j \in J} a_{jsv}^{\text{MAX}} x_{js} \geq b_{sv}^{\text{GEQ}}, \quad \text{with } a_{jsv}^{\text{MAX}} = \max_{s.t.} (9) - (11) a_{jsv}^{\text{GEQ}} w_{js}, \quad v \in V, \quad (14)$$

are valid inequalities for [P].

Proof. Consider the feasible set described by (7)–(12). Using (11) gives that (7) and (8) can be equivalently stated as

$$\sum_{j \in J} a_{jsu}^{\text{LEQ}} w_{js} x_{js} \leq b_{su}^{\text{LEQ}}, \quad u \in U, \quad \text{and} \quad \sum_{j \in J} a_{j sv}^{\text{GEQ}} w_{js} x_{js} \geq b_{sv}^{\text{GEQ}}, \quad v \in V.$$

Since $a_{jsu}^{\text{MIN}} x_{js} \leq a_{jsu}^{\text{LEQ}} w_{js} x_{js}$, $a_{j sv}^{\text{MAX}} x_{js} \geq a_{j sv}^{\text{GEQ}} w_{js} x_{js}$, $j \in J$, $s \in S$, $u \in U$, $v \in V$, the result follows. $\square$

Note that an equality constraint can be represented by its two corresponding inequality constraints and the results of the proposition can be applied to these.

3 Scheduling Problems with Implication Structure

We study two applications in our preliminary investigations: cumulative facility scheduling and single-facility scheduling with a segmented timeline, using two different objectives, both to minimise assignment cost and to minimise makespan. In the following we state a model that captures both applications, in line with model [P]. Let J index the set of jobs and S index a set of facilities. The variables are defined as in [P]. For $j \in J$, $s \in S$, let the processing time and resource consumption per time unit of job j in facility s be p_{js} and c_{js} , respectively. Let the domain $\mathcal{D}_{js}^s = \{r_{js}, \dots, d_{js} - p_{js}\}$, where r_{js} is the release time and d_{js} is the deadline for job $j \in J$ in facility $s \in S$. In facility $s \in S$ the maximum allowed resource consumption per time unit is C_s .

The model is given by

$$\begin{aligned} \min \quad & h(x) + g(y), \\ \text{s.t.} \quad & \sum_{s \in S} x_{js} = 1, & j \in J, \\ & \text{CUMULATIVE}((y_{js}, p_{js}, c_{js})_{j \in J | x_{js}=1}, C_s), & s \in S, \\ & y_{js} \in \{r_{js}, \dots, d_{js} - p_{js}\}, & j \in J, s \in S \mid x_{js} = 1, \\ & y_{js} = 0, & j \in J, s \in S \mid x_{js} = 0, \\ & x_{js} \in \{0, 1\}, & j \in J, s \in S, \end{aligned}$$

where the first and second constraint corresponds to (2) and (3), respectively, in [P]. Application-specific models are obtained as follows: For cumulative facility scheduling, release times and deadlines are subproblem independent and replaced by $r_{js} = r_j$ and $d_{js} = d_j$, $j \in J$, $s \in S$. For single-facility scheduling, $p_{js} = p_j$, $j \in J$, $s \in S$. There is also a disjunctive constraint in the subproblem instead of the cumulative constraint, which is obtained by fixing $c_{js} = 1$, $C_s = 1$, $j \in J$, $s \in S$, in the cumulative constraint. When minimising assignment cost the objectives are $h(x) = \sum_{j \in J, s \in S} f_{js} x_{js}$ and $g(y) = 0$, where f_{js} is the cost of assigning job $j \in J$ to facility $s \in S$, and when minimising makespan they are $h(x) = 0$ and $g(y) = \max_{j \in J, s \in S} y_{js} + p_{js}$. For single-facility scheduling,

$h(x) = \sum_{j \in J, s \in S} 2p_j(1 - x_{js})$ which includes a penalty of $2p_j$ if job $j \in J$ is unscheduled. The decomposition of the problem follows that of [P].

A commonly used MIP formulation for the cumulative constraint is to make a time-indexed formulation and introduce capacity constraints for each time step. From this, a relaxation is obtained by aggregating constraints over time intervals. A derivation of this relaxation and its use in Proposition 1 is as follows.

For $s \in S$, let $r_s^{\min} = \min_{j \in J} r_{js}$ and $d_s^{\max} = \max_{j \in J} d_{js}$ be the earliest release time and latest deadline, respectively. Let $T_s = \{r_s^{\min}, \dots, d_s^{\max}\}$ be the set of integer time steps, $s \in S$. Introduce the variables $w_{js} = (w_{jst})_{t \in \{r_{js}, \dots, d_{js} - p_{js}\}}$, where w_{jst} indicates if job j starts at time t in subproblem s or not, $j \in J$, $s \in S$. Its domain, for $s \in S$, $t \in \{r_{js}, \dots, d_{js} - p_{js}\}$ is defined as $w_{jst} \in \{0, 1\}$, $j \in J \mid x_{js} = 1$ and $w_{jst} = 0$, $j \in J \mid x_{js} = 0$, corresponding to (10) and (11) in Proposition 1. For $s \in S$, the capacity for time step $t' \in T_s$ is respected if

$$\sum_{j \in J} c_{js} \sum_{t=t'-p_{js}+1}^{t'} w_{jst} \leq C_s, \quad \sum_{t \in \{r_{js}, \dots, d_{js} - p_{js}\}} w_{jst} = 1, \quad j \in J \mid x_{js} = 1.$$

Let the latter constraint correspond to (9) in Proposition 1 and note that a relaxation of the former constraint can be obtained by summing such constraints over a subset of time steps. Introduce U_s to index a set of time intervals for $s \in S$ and, for $u \in U_s$, let $T'_{su} = \{t_{su}^{\text{START}}, \dots, t_{su}^{\text{END}}\} \subseteq T_s$ denote a set of consecutive time steps. The linear formulation we use as Constraint (7) in Proposition 1 is

$$\sum_{j \in J} c_{js} \sum_{t'=T'_{su}} \sum_{t=t'-p_{js}+1}^{t'} w_{jst} \leq C_s (t_{su}^{\text{END}} - t_{su}^{\text{START}}), \quad u \in U_s.$$

To compute $a_{jsu}^{\min}$ in Proposition 1, note that a lower bound on the number of times the sum $\sum_{t=t'-p_{js}+1}^{t'} w_{jst}$ can take the value one is obtained by considering the number of time steps in the interval between r_{js} and d_{js} that are outside of the interval $t_{su}^{\text{START}}, \dots, t_{su}^{\text{END}}$ and assume that those will be used. It then follows from Proposition 1 that, given a $u \in U_s$, $s \in S$

$$\begin{aligned} a_{jsu}^{\min} &= \min_{\substack{t \in \{r_{js}, \dots, d_{js} - p_{js}\} \\ w_{jst} \in \{0, 1\} \text{ if } x_{js} = 1 \\ w_{jst} = 0, \text{ if } x_{js} = 0}} \sum_{t'=T'_{su}} \sum_{t=t'-p_{js}+1}^{t'} w_{jst} \\ &= \max \{0, p_{js} - \max \{0, t_{su}^{\text{START}} - r_{js}\} - \max \{0, d_{js} - t_{su}^{\text{END}}\}\}, \end{aligned} \quad (15)$$

and

$$\sum_{j \in J} c_{js} a_{jsu}^{\min} x_{js} \leq C_s (t_{su}^{\text{END}} - t_{su}^{\text{START}})$$

is a valid inequality for our scheduling problem.

The derived inequalities relate to the ones introduced in Sect. 1.3 as follows. First, note that the energy and segment relaxations only include jobs for which it holds that $t_{su}^{\text{START}} \leq r_{js}$ and $d_{js} \leq t_{su}^{\text{END}}$, while (15) can also handle partial contributions of a job. Choosing the set U_s , $s \in S$, so that the time intervals are between all release time and deadline pairs and considering only $j \in J$ such that $t_{su}^{\text{START}} \leq r_{js}$ and $d_{js} \leq t_{su}^{\text{END}}$ hold, our valid inequality coincides with the energy relaxation. Using a similar argument, but with the set U_s chosen so that a time interval is a segment, our valid inequality becomes the segment relaxation.

4 Implementation

The concepts presented in this paper are implemented using the Benders decomposition framework of SCIP [11]. A branch-and-cut version of Benders decomposition is applied using the default Benders decomposition plugin, where the subproblems are solved as SCIP instances. A feature of SCIP is that general implementations of both classical Benders and LBBD-based cuts are available. Classical cuts are generated from the LP relaxation of the subproblem, while the LBBD-based no-good cuts are generated from the solution to the CIP.

Our implementation extends SCIP with a variant of a CUMULATIVE constraint handler. The new constraint handler, termed CUMULATIVEBENDERS, permits jobs to be optionally selected in a cumulative constraint to consume resources. Prior to solving each subproblem—given a master assignment solution—CUMULATIVEBENDERS adds a CUMULATIVE constraint that includes only the jobs assigned to this subproblem.

The CUMULATIVEBENDERS constraint is important regarding the implementation of Constraints (7)–(12). The valid inequalities for both the master and subproblem are implemented as part of the CUMULATIVEBENDERS constraint, which are added during the initialisation stage immediately prior to presolving each problem. As a result, the CUMULATIVEBENDERS constraint handler provides a general method to add subproblem relaxations for the CUMULATIVE and DISJUNCTIVE constraints when applying Benders decomposition. For our preliminary experiments in this paper, we chose the sets U_s , $s \in S$, and the jobs so that we use the inequalities corresponding to the energy and segment relaxations.

5 Computational Results and Conclusions

The computational experiments assess the effect of using the generic relaxation of Proposition 1 in the master or subproblem when applying LBBD to the problems in Sect. 3. The problems are solved by SCIP 10.0 using Gurobi Optimizer 11.0 as the LP solver. The experiments were executed on a compute node comprising two Intel Xeon Gold 6130 processors (32 cores, 2.10 GHz) and 376 GB of RAM. Each instance was solved using 8 CPU cores with a time limit of 20 min. For cumulative facility scheduling, the 335 instances presented in [6] are used. For single-facility scheduling, the 480 instances from [12], based on [1], are used.

The results presented in Fig. 1 show that adding generically-derived valid inequalities to the master problem exhibit similar performance improvements as problem-specific subproblem relaxations. The explanation of this performance is that the generically-derived valid inequalities focus on eliminating master problem solutions that induces infeasible subproblems—thus, reducing the number of LBBD-based cuts that are generated. An exception is Fig. 1b, where the subproblems are always feasible and hence there are no master solutions to eliminate. In this case, objective-based valid inequalities are potentially better suited to improve computational performance. Overall, when subproblems become infeasible, the generically-derived valid inequalities are computationally effective.

Figs. 1c and 1d shows that adding valid inequalities to the subproblem does not greatly improve computational performance. This is due to the valid inequalities being designed to eliminate infeasible subproblems, and not necessarily strengthening the LP relaxation of the subproblem. In most instances, the valid inequalities do not sufficiently strengthen the LP relaxation of the subproblem to generate classical Benders optimality and feasibility cuts—hence, relying only on LBBD-based cuts to eliminate infeasible or suboptimal solutions.

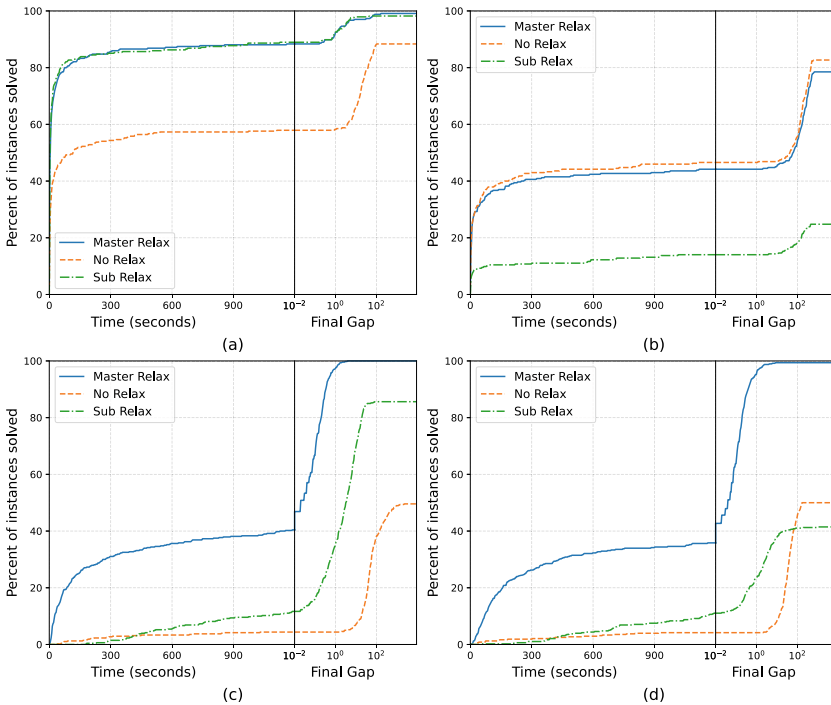


Fig. 1. Percentage of instances solved to optimality within 1200s and the remaining optimality gap thereafter. Cumulative facility scheduling: (a) total cost, (b) makespan. Single-facility scheduling: (c) total cost, (d) makespan.

In conclusion, a framework for deriving valid inequalities is a valuable contribution to improve the general performance of LBBD. As confirmed by these results and from [10], valid inequalities based on the constraints of the problem are very effective at eliminating master solutions which induce infeasible subproblems. Extending the concepts in this paper to derive objective-based valid inequalities is a necessary step in the development of a general LBBD solver.

Acknowledgements. The work of S. Rajendran and E. Rönnberg was mainly supported by the Excellence Center at Linköping Lund in Information Technology (ELLIIT), and partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP). Computational experiments were performed on resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS).

Disclosure of Interests. The authors have no competing interests.

References

1. Coban, E., Hooker, J.N.: Single-facility scheduling by logic-based benders decomposition. *Ann. Oper. Res.* **210**, 245–272 (2013)
2. Fachini, R.F., Armentano, V.A.: Logic-based benders decomposition for the heterogeneous fixed fleet vehicle routing problem with time windows. *Comput. Ind. Eng.* **148**, 106641 (2020)
3. Harjunkski, I., Grossmann, I.E.: Decomposition techniques for multistage scheduling problems using mixed-integer and constraint programming methods. *Comput. Chem. Eng.* **26**, 1533–1552 (2002)
4. Hojny, C., et al.: The SCIP optimization suite 10.0. Technical report, Optimization Online, November 2025. <https://optimization-online.org/2025/11/the-scip-optimization-suite-10-0/>
5. Hooker, J.: *Logic-Based Benders Decomposition - Theory and Applications*. Springer Nature, Switzerland (2024)
6. Hooker, J.N.: Planning and scheduling by logic-based Benders decomposition. *Oper. Res.* **55**, 588–602 (2007)
7. Jain, V., Grossmann, I.E.: Algorithms for hybrid MILP/CP models for a class of optimization problems. *INFORMS J. Comput.* **13**(4), 258–276 (2001)
8. Karlsson, E., Rönnberg, E.: Strengthening of feasibility cuts in logic-based benders decomposition. In: Stuckey, P.J. (ed.) *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pp. 45–61. Springer International Publishing, Cham (2021)
9. Karlsson, E., Rönnberg, E.: Logic-based Benders decomposition with a partial assignment acceleration technique for avionics scheduling. *Comput. Oper. Res.* **146**, 105916 (2022)
10. Lam, E., Gange, G., Stuckey, P.J., Van Hentenryck, P., Dekker, J.J.: Nutmeg: a MIP and CP hybrid solver using branch-and-check. *SN Oper. Res. Forum* **1**, 22:1–22:27 (2020)
11. Maher, S.J.: Implementing the branch-and-cut approach for a general purpose Benders’ decomposition framework. *Eur. J. Oper. Res.* **290**(2), 479–498 (2021)

12. Saken, A., Karlsson, E., Maher, S., Rönnberg, E.: Computational evaluation of cut-strengthening techniques in logic-based benders' decomposition. *Oper. Res. Forum* **4** (2023)
13. Thorsteinsson, E.S.: Branch-and-check: a hybrid framework integrating mixed integer programming and constraint logic programming. In: Walsh, T. (ed.) *CP 2001*. LNCS, vol. 2239, pp. 16–30. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-45578-7_2